

CS242: System Software Lab

**Versioning System and
Debugger**

Tut02 : 19th Aug 2024

Dr. A. Sahu

Dept of Comp. Sc. & Engg.

Indian Institute of Technology Guwahati

Outline

- Simple VCS:
 - SCCS
- Versioning System
 - VCS offline
 - VCS online: git-hub
- Debugger
 - Command line Debugger: gdb
 - Command line Debugger: ddd

What is Source Code Management

- SCM : Also known as revision or source control
- So, why should you use one?

Old-Fashioned Version Control

- The simplest version control:
 - Every so often,
 - make a copy of your code,
 - give it a unique name (usually a version number)
 - Place it somewhere safe
- You should also make a note (notebook, README file)
 - describing the features of this version and how it differs from past versions

What is Version Control?

- Over time files evolve and change.
- Tracking these changes can be difficult,
 - Since the changes stem from different project teams.
- Version control allows users
 - to maintain the integrity of files that are subject to change.

Why Do We Care?

- Version control systems (VCS) provide
 - Users the ability to serialize changes to a given file.
- Most such systems also allow
 - Users to revert to an earlier form of a file.
- Anything can be stored in good VCS
 - including source code, documents, images, and binaries.

Version Control

- Software development never stops!
 - This is especially true of scientific code which we're always improving
- We need a way to keep track of what changes we've made
 - Need to go back if we break something
 - Important for reproducibility

Version Control Systems

- There are many systems that automate version control. These,
 - Store your code, often more efficiently than lots of copies
 - Document changes
 - Provide easy access to old versions, and comparisons

SCCS : simple vcs

- Install sccs by `$sudo apt-get install cssc`

```
$mkdir sccs_test
```

```
$cd sccs_test
```

```
$echo 'hallo' > quit.c
```

```
$mkdir SCCS
```

```
$sccs create quit.c
```

```
$sccs edit quit.c
```

```
$echo 'bye' >> quit.c
```

```
$sccs delta quit.c
```

CVS

- CVS is the standard for version control
 - Available on all systems (even in MS Windows)
- Stores your files in a special place : **repository**
 - Stores initial files, then changes from them, so very efficient
- Projects are bundled into modules
 - Repository can hold lots of modules
- To work on your code,
 - Check it out from the repository
 - Make changes
 - Check the code back in, describe changes

CVS for Project submission

- Terminology used
 - Repository
 - Server
 - Client
 - Working Directory
- Some of the options available to connect to the repository
 - Command-line using Putty
 - Through Eclipse IDE

CVS Command

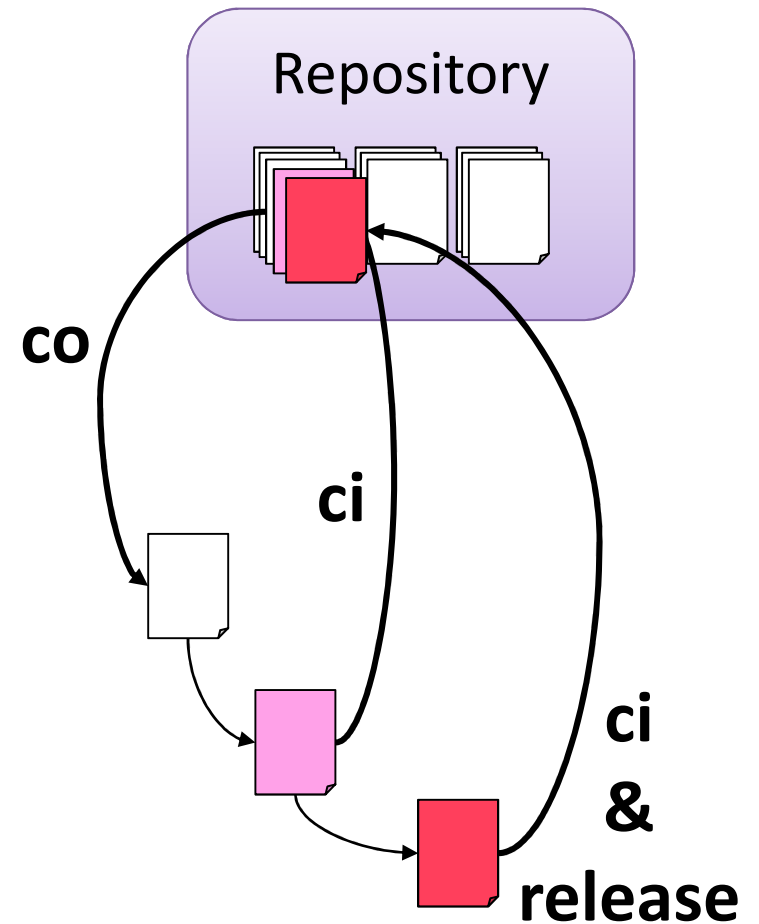
- The general form of CVS commands is:
cvs [cvs-options] command [command-options-and-arguments]
- All CVS commands start out with “cvs”
- Commands may also have flags and/or arguments which modify their behavior
- For a more help...
 - General help: cvs --help
 - List of commands: cvs --help-commands

CVS

- CVS was designed for large, corporate projects
 - Allows several users to work on a single module
 - Forces the users to document the changes they made
 - Forces the users to reconcile their changes with each other
 - Possible to put CVS servers on the net, allowing lots of people to work on code (e.g. Linux, GNU)
- For small-scale science
 - CVS keeps track of what you've done
 - Allows you to try new things without ruining what you've done
 - Allows you to reproduce old results--exactly!

CVS Concepts

- Repository--central home of modules (like a library)
- Module--a collection of associated files (like a book)
- Check Out--get a copy of a module
 - Implies that you will make changes
- Check In--sends changes to repository
 - CVS forces you to describe changes
- Release--you are done developing the module



What is CVS?

- Concurrent Versioning System (CVS) is one of the earlier SCM systems which gained wide adoption
- Advantages
 - Open source
 - Easy to install and use
 - **`$sudo apt-get install cvs`**
 - Simple command line client
 - Wide integration in a lot of development tools: eclipse, kdevelop

Basic CVS Commands

- checkout
 - Pull resources from the repository and create a working copy
- add
 - place a resource under version control
- update
 - Pull down changes from the repository into your working copy
- Commit
 - Check files into the repository

CVS Command

- The general form of CVS commands is:

```
cvcs [cvcs-options] command [command-options-and-arguments]
```

- All CVS commands start out with “cvcs”
 - Commands may also have flags and/or arguments which modify their behavior
- For a more help...
 - General help: `cvcs --help`
 - List of commands: `cvcs --help-commands`

Repository

- All version control systems work based on a *repository*.
- The repository holds a copy of each file under control of the version control system.
- The repository generally maintains an initial copy of each file along with a log of all subsequent changes made to that file.

Adding Files to the Repository

- Not all of the files on your system are automatically version controlled when a version control system is installed.
- To “version” a file, you must explicitly place that file into the repository.
- A file under control of the version control software is commonly called an *archive*.

Checking Out Archives From the Repository

- Once a file has been placed under revision control, you must check it out of the repository in order to change it.
- This ensures that one user doesn't inadvertently overwrite the changes made by another user.
- This is called *checking out* an archive.

Checking Out Archives From the Repository

- Every version control system is essentially a library with each archive a book.
 - At any given time, one and only person can have possession of a book.
 - In version control systems, only the user with possession of an archive can change that archive.
- It is also possible to check out read-only copies of archives.

Checking In Files to the Repository

- Once you have an archive checked out, you can make changes to it.
- Once you have completed the changes, it must be checked back into the repository so that it is again available to the other users of the version control system.
- This is called *checking in* an archive.

Unlocking a Revision

- Sometimes we check out an archive
 - Intending to make changes to it, but then discover we don't really have to.
- We don't want to check in an archive
 - that hasn't changed.
- Most version control systems provide
 - the ability to *unlock* a checked out archive without actually checking it in.

Revisions

- When you save your changes using a VCS
 - The new version of the archive is given a new revision number.
- It isn't necessary to know how the revision numbers are generated
 - (it's a form of magic), but you should know
 - that they represent the changes in an archive over time.

Checking Out Archives by Revision

- Sometimes we want to check out an archive by a specific revision number.
- This is useful if we need to revert to an earlier code base in order to diagnose a bug.
- Be careful! If you make changes to earlier revision of an archive and
 - Check it back in, you lose all of the changes in the later revisions of that archive.

CVS Example

- **\$export CVSROOT='/home/asahu/cvs_root'**
 - Directory for CVS source code repository
 - setenv CVSROOT '/home/linux/cvs_root' in .bashrc
- **\$cvs -d /home/asahu/cvs_root init**
 - Creating an repository
- **\$cvs import -m "CVS START" cvs_file myfile start**
 - add a project named “myfile” with a description “CVS START”.
- **\$cvs checkout cvs_file**
 - To work on a project, you need to create a local copy of it from the repository

CVS Example Contd..

- `$mkdir cvs_file_1`
- `$cvs add cvs_file_1`
 - New file/dir named “cvs_file_1” that you want to add
- `$cvs commit myfile`
 - commit the changes made to the file “myfile”
- `$cvs update`
 - ensure your local working copy is up to date with the repository
 - synchronizes your working directory with the latest versions of files in the repository.
- `$cvs remove myfile`
 - remove the file “myfile” from the repository

GIT

- `$git config --global user.name "cs242-test"`
- `$git config --global user.email "cs242@iitg.ac.in"` A
- `$mkdir myproject ; $ cd myproject`
- `$git init`
 - Initialized empty Git repository in
/home/asahu/myproject/.git/
- `$git status`
 - No commit yet
- `$vim hello.c`
- `$git add hello.c`
- `$git commit -m "First release of Hello World!"`²⁸

GIT

- `$git status --short`
 - M hello.c
- `$git commit -a -m "Updated hello.c with a new line"`
 - Added a newline and commit
- `$git fetch myproject`

Reference

- Good tutorial for cvs

<http://www.gnu.org/software/cvs/manual/cvs.html>

Thanks